

FastAPI Developer Coding Assignment

Outcome-based backend assignment: authentication, roles, task assignment, caching, and email-based 2FA

Objective. Build a FastAPI backend for a simple task management workflow. The assignment is intentionally outcome-driven: we should be able to run a fixed workflow and validate the implementation from the final API response.

AI usage. Candidates may use AI tools such as ChatGPT, Cursor, Claude, or GitHub Copilot. We care about the working result, code quality, and whether the candidate can explain their implementation.

Technical Requirements

- Python 3.11+ with FastAPI, SQLAlchemy, Pydantic, Alembic, Pytest, and JWT authentication.
- PostgreSQL preferred; SQLite is acceptable for local development if documented.
- Redis preferred for caching; in-memory cache is acceptable if limitations are documented.
- Email delivery may use SMTP/provider integration, console logs, or a development email log table.
- Optional: Docker, Ruff/Black, GitHub Actions.

Required Workflow

Step	Action	Expected Outcome
1	Create two users	Create an Admin user and a staff user named James Bond.
2	Login as Admin	Admin login starts email-based 2FA. After verifying the code, return a JWT token.
3	Create 5 tasks	Using the Admin token, create exactly five tasks.
4	Assign tasks	Assign exactly three of the five tasks to James Bond.
5	Login as James Bond	James Bond login also uses email-based 2FA and returns a JWT only after verification.
6	James Bond attempts to create task	This request must fail with 403 Forbidden / not authorised.
7	View my tasks	James Bond calls /tasks/view-my-tasks and receives exactly three assigned tasks.

8	Validate caching	Calling /tasks/view-my-tasks a second time should show cache.hit = true.
---	------------------	--

Suggested API Endpoints

POST /seed/users	Create Admin and James Bond for validation
POST /auth/login	Start login and send 2FA code
GET /dev/email-logs/latest	Read latest development 2FA email/code
POST /auth/verify-2fa	Verify code and receive JWT token
POST /tasks	Create task (admin only)
POST /tasks/assign	Assign selected tasks to James Bond (admin only)
GET /tasks/view-my-tasks	Return current user's assigned tasks (cached)

Note: The exact route names can differ, but the candidate must document them clearly and support the same workflow.

Seed / Setup Requirement

The candidate should provide a simple way to create the validation dataset. This can be a seed endpoint or a standalone script. A script is preferred for repeatable local setup.

```
python scripts/seed_demo.py --reset
```

Seed data must create:

- Admin User / admin@example.com / Password123! / admin
- James Bond / jamesbond@example.com / Password123! / staff

Key API Behaviour

- Admin can create tasks. Staff cannot create tasks.
- Admin can assign tasks. Staff cannot assign tasks.
- James Bond can only view tasks assigned to James Bond.
- The system must return exactly three tasks from /tasks/view-my-tasks after assignment.
- Responses must be generated from database records, not hardcoded data.

Two-Factor Authentication

- Login with email/password should not immediately return a JWT token.
- The login endpoint should create a one-time verification challenge and trigger an email.
- For local validation, the latest email/code may be available through a development-only email log endpoint.

- The verify endpoint should return the JWT token only when the correct code is supplied.
- Codes must expire after 5 minutes, be single-use, and reject incorrect or reused codes.
- If persisted, verification codes should not be stored in plain text.

Caching Requirement

- Cache the /tasks/view-my-tasks response per user.
- Use a cache key such as user_tasks:{user_id}.
- First request should return cache.hit = false; second request should return cache.hit = true.
- When tasks are assigned or updated, invalidate the relevant user task cache.
- Redis is preferred. In-memory cache is acceptable for local development if documented.

Expected Final Validation Response

After completing the workflow, James Bond should call:

```
GET /tasks/view-my-tasks
```

The response should be similar to:

```
{
  "user": {
    "email": "jamesbond@example.com",
    "role": "staff"
  },
  "tasks": [
    { "id": "...", "title": "...", "status": "todo", "priority":
"high", "assigned_to": "jamesbond@example.com" },
    { "id": "...", "title": "...", "status": "todo", "priority":
"medium", "assigned_to": "jamesbond@example.com" },
    { "id": "...", "title": "...", "status": "todo", "priority":
"low", "assigned_to": "jamesbond@example.com" }
  ],
  "summary": {
    "total_assigned_tasks": 3
  },
  "cache": {
    "hit": false
  }
}
```

Calling the same endpoint again should return the same tasks with cache.hit = true.

Minimum Data Model

- User: id, full_name, email, hashed_password, role, created_at, updated_at.

- Task: id, title, description, status, priority, created_by_id, assigned_to_id, created_at, updated_at.
- LoginChallenge or equivalent: challenge id, user id, hashed code, expiry, used flag.
- EmailLog or equivalent for local/dev validation of the 2FA email event.

Testing Expectations

- Admin and James Bond can be created.
- Login creates a 2FA challenge and does not immediately return a JWT.
- Correct 2FA code returns a JWT; incorrect, expired, and reused codes fail.
- Admin can create five tasks.
- Admin can assign exactly three tasks to James Bond.
- James Bond cannot create tasks and receives 403 Forbidden.
- James Bond can view exactly three assigned tasks.
- Caching works: first view-my-tasks call is a miss, second is a hit.
- Assigning/updating tasks invalidates the relevant user cache.

Submission Requirements

- GitHub repository with application code.
- README.md with setup, migration, seed, run, workflow, and test instructions.
- AI_USAGE.md explaining which AI tools were used and what was manually changed.
- Database migrations or documented schema setup.
- Tests for the required workflow.
- Example final /tasks/view-my-tasks response in the README.

Allowed Assumptions

- There are only two roles: admin and staff.
- Only admin users can create and assign tasks.
- Staff users can only view their assigned tasks.
- Task status can default to todo.
- Priorities can be low, medium, high, or urgent.
- A development email log endpoint is acceptable instead of real email delivery.
- In-memory cache is acceptable if Redis is not used, but the README must explain its limitations.
- The seed/reset workflow only needs to be safe for local development.
- The candidate may choose SQLite or Postgres, as long as setup is clearly documented.
- The final validation response is more important than matching the exact internal implementation.

Evaluation Criteria

- The required workflow works end-to-end.

- Authentication and 2FA are implemented sensibly.
- Role-based permissions are enforced.
- James Bond sees exactly three assigned tasks and cannot create tasks.
- Caching behaviour is visible and cache invalidation works.
- Code is clean, maintainable, and testable.
- The candidate can explain their implementation and AI usage.